Base de datos 1

Jaime Nazar Anchorena

Contents

Unidad 1: Caracteristicas de un DBMS **5**
Se puedde considerer que una Planilla Excel es una base de datos? . . . 5
Sistema de manejo de base de datos(DBMS) . . . 5
Diferencias entre DMBS y un sistema de procesamiento de archivos . . . 5
Modelo de datos . . . 6
Categorias . . . 6
Arquitecturas de tres niveles . . . 6
Arquitecturas tipicas para un DBMS . . . 6

Unidad 2: Modelo conceptual, modelo de entidad-relacion **7**
Modelo de Entidad/Relacion y su extencion . . . 7
Pasos de un buen disenador . . . 7
Entidades(entity type) . . . 7
Relaciones(Relationship type) . . . 7
Atributos . . . 8
Modelo ER extendido agrega jerarquias . . . 9
Tipos de jerarquias . . . 9
Semantica de relaciones . . . 9

Unidad 3: Modelo de representacion/implementacion: Modelo relacional **9**
Un poco de historia . . . 9
Modelo relacional . . . 10
Atributo . . . 10
Esquema . . . 10
Relacion . . . 10
Relacional vs Entidad/Relacion . . . 11
Notacion para el modelo relacional . . . 11
Restricciones del modelo relacional . . . 11
Restricciones de domino . . . 11
Restricciones de clave . . . 11
Restriccion de integridad de entidad . . . 12
Restriccion de integridad referencial . . . 12
Restriccion de dependencias funcionales . . . 12
... . . . 12

… 12
Lenguajes del DBMS 12
Operacion de insercion 12
Operacion de borrado 12
Operacion de modificacion 13
Pasaje de entidad/relacion a relacional 13
Pasos para un buen disenador 13
Mapeo de entidades fuertes 13
Mapeo de entidades debiles 13
Mapeo de atributos multivaluados 13
Mapeo de relaciones 13
Mapeo de Jerarquia generalizacion 15

Unidad 4: Lenguajes de consulta: Algebra y calculo relacional 15
Algebra 15
Fundamentales 15
Derivados 16
Definicion: Relaciones compatibles 17
Calculo relacional 17
Calculo relacional de tuplas 17
Calculo relacional de dominio 19

Unidad 5: Languaje de consulta SQL 20
Lenguaje de definicion de datos(DDL) 20
Tipos 20
Dominios 21
Creacion de tablas persistentes 21
Destruccion de tablas 21
Cambio de esquema de tablas 21
Lenguaje de manipulacion de datos(DML) 21
Insercion de tuplas 21
Borrado de tuplas 22
Actualizado de tuplas 22
Lenguaje de manipulacion de datos (DML) 22
Ambiguedades y soluciones 22
Expresiones en la clausura `FROM` 23
Consultas anidadas 23
Expresiones en la clausura `WHERE` 23
Operadores sumarizacion 24
Transformacion de expresiones 25
Modificacion 25
Vistas 25
Permisos 26
Es el algebra relacional equivalente al calculo relacional seguro? 26
Clausura 26
Clausula `WITH` 27

Unidad 7: Teoria de normalizacion **27**
Definicion: Dependencia funcional . . . 28
Redefinicion: Superclave . . . 28
Redefinicion: Clave candidata . . . 28
Dependencia funcional . . . 28
Clausura de F . . . 28
Axiomas de Armstrong . . . 28
Reglas de inferencia . . . 29
Clausura de un conjunto de atributos . . . 29
Algoritmo de Tableaux . . . 29
Definicion: Descomposicion sin perdida de dependencias . . . 29
Algoritmo para calcular la proyeccion de F^+ sobre un subesquema de R(el algoritmo) . . . 29
Tema clase 6/10 . . . 29
Repaso . . . 29
Mal diseno . . . 30
Enfoque analitico . . . 30
Proceso de normalizacion . . . 30
Basados en dependencias funcionales . . . 30
Basados en dependencias multivaluadas . . . 31
Basados en Dependencias de junta . . . 31

Unidad 8: SQL avanzado **31**
Persistent Stored Module(PSM) . . . 32
PostgreSQL . . . 32
Como se manejan los errores . . . 32
Algoritmo para manejar excepciones . . . 33
Cursor . . . 33
Implicito . . . 33
Explicito . . . 33
Codigo activo . . . 34
Restricciones de dominio . . . 34
Restricciones avanzadas . . . 34
Triggers . . . 34
Trigger PSM . . . 35
Restricciones adiciones . . . 35

Unidad 9: Programacion embebida - C **36**
Host variable . . . 36
Conexion con DB . . . 36
Manejo de errores . . . 36
Forma implicita antigua . . . 36
Forma implicita moderna . . . 36
Forma whenever . . . 36
Variable indicator . . . 36
Cursor explicito . . . 37
Cosas a tener en cuenta . . . 37

Claves ficticias . . . 37
Buena tecnica . . . 37
Dos tablas se autoreferencian . . . 38
Transacciones y programacion embebida . . . 38
Problemas que soluciona la aislacion . . . 38

Unidad 9: Programacion embebida - Java **39**
Formas de cargar drivers . . . 39
Primera forma . . . 39
Segunda forma . . . 39
Tercera forma . . . 39
Cuarta forma . . . 39
Tipos de drivers . . . 39
Conseguir estancia . . . 39
CreateStatement . . . 40
PreparedStatement . . . 40
Nuevas versiones de JDBC . . . 40
Pool de conexiones . . . 40

Unidad 1: Caracteristicas de un DBMS

Significa *Data Base Management System*. Las bases de datos se utilizan en todos lados, incluso es normal que una organizacion manege mas de una base de datos.

Se define como una coleccion de datos relacionados, que cumplen con las siguientes propiedades:

- Representan **algun aspecto del mundo real(mini-universo)**, no todo(por ejemplo, que idiomas hable un alumno) pues depende del objectivo
- Tienen **coherencia logica** con algun significado inherent Se disena, contruye y se completa con datos para un **proposito especifico**. Pueden tener diferentes grupos de usuarios cada uno con **intereses distintos** y ademas cuenta con cierta aplicaciones preconcebidas para lograr cierto objetivo.

La base de datos recide en disco, se trae de a **chunks** para procesarlos.

Una base de datos ofrece una **abstraccion** sobre los datos que almacena, por cuandto oculta detalles de su almacenamiento y manejo.

Una base de datos no posee limite en la cantidad de informacion que puede manipular, sin importar la performance la base de datos no debe pedir al usuario que debe guardar la informacion en otro lugar pues se quedo sin espacio.

Se puedde considerer que una Planilla Excel es una base de datos?

No, debido a que:

- No se pueden definir perfiles de usuarios que accedan a determinada parte de la planilla o tengan distintos tipos de permisos. *En un excel todos pueden leer todo.*

- Aunque pueda almacenar datos, si el volumen es muy grande el recupero de la informacion en forma eficiente se vuelve conflictiva.

Basicamente, es un archivo, no una base de datos.

Sistema de manejo de base de datos(DBMS)

Coleccion de programas que permiten crear y mantener una base de datos.

- Crearla implica **definir los tipos** de datos, estructura y **restricciones** de los datos que se almacenen.
- Mantenerla implica **actualizar sus datos** para que reflejen los cambios del milti-universo que representa, generar reportes y poder **consultar** la informacion que posee almacenada.
- El software de DBMS no responde a fines especificos, a diferencia de la base de datos.

Diferencias entre DMBS y un sistema de procesamiento de archivos

- **Autodescripcion(catalogo)**: En Java no habia un repo aparte donde se declaraba la informacion, sino que estaba todo embebido en el codigo(strings, constantes, etc.).
- **Independencia entre datos y programas(mejor que TAD)**: La base de datos puede cambiar en el tiempo, mientras que el programa puede no cambiar nada. Significa que hay niveles por lo cual los niveles mas bajos pueden cambiar sin que los demas se den cuenta. Esto no se puede hacer un un TAD.
- Ofrece vistas a distingtos grupos de usuarios(views)

- **Integridad de datos**: Cada vez que cambie algo, en una base de datos se valida si lo que cambie cumple con todas las reglas. Si no cumple, se notificara al usuario para que se cumpla si o si.
- **Manejo de concurrencia**: No puede haber problemas de concurrencia, si dos usuarios cambian el dato al mismo tiempo, debe ser manejado eso. Debe soportar varios usuarios en simultaneo.
- **Manejo transaccional(atomicidad)**: Se maneja como un todo, *todo o nada*. Si se corta la luz o se corta el servicio, no puede quedar inconsistente, debe volver al estado anterior o guardar el estado para que el usuario termine la modificacion.

  Nota: Las ultimas dos reglas son muy dificiles de programar, no es trivial(puede llevar anos). Tambien son la mayor diferencia, son la razon por la cual se elije usar una base de datos.

Modelo de datos

Conjunto de conceptos que se utilizan para describir la estructura(tipos de datos, relaciones y restricciones) de una base de datos.

```
Modelo -> Abstraccion
```

La mayoria de los modelos de datos ofrecen tambien un conjunto de operaciones basicas(built-in) para el recupero y actualizacion de la base de datos, e inclusivo algunas permiten especificar omportamientos o sea operaciones definidas por el usuario.

Como usuario solo quiero saber como va a quedar, no me interesa todos los detalles tecnicos de coo se llego a ello(como seria contruir una casa, por ejemplo). *Modelo siempre es abstraccion, sino no es modelo.*

Categorias

- **Conceptual**, alto nivel semantico, basado en entidades, objetos.
- **De representacion o implementacion**, Modelo de red, jerarquico, relacional.
- **Fisico**, de bajo nivel o interno(estructuras, almacenamiento).

El mejor, con el que vamos a trabajar, sera el modelo basado en entidades.

Arquitecturas de tres niveles

Definidos por ANSI. Para base de datos relacionales.

- **Nivel interno o vista interna**: Posee un esquema interno que describe la estructura de almacenamiento fisico
- **Nivel conceptual o visual conceptual**: Posee une squema conceptual que describe la estructura de la base de datos completa.
- **Nivel externo o vista externa**: Posee varias vistas de usuarios.

Se debe cambiar de uno a otro sin afectar los otros.

Nota: Es el lenguaje defacto para todas las bases de datos relacionales que hay.

Arquitecturas tipicas para un DBMS

- Cliente-Servidos centralizado 2-tier -...

Unidad 2: Modelo conceptual, modelo de entidad-relacion

Las bases de datos son un pedazo de sistemas de informacion que manejan cualquier organizacion con el objetivo de tomar decisiones y registrar sus procesos.

Tan complejo es poder manejar volumen de informacion hetereogenea que cada vez es mayor la tecnologia emergente que se va incorporando a las empresas con el fin de permitir que la misma sea mas organizada.

La informacion puede usarse en la medida en que se encuentre **pre-procesada**.

Modelo de Entidad/Relacion y su extencion

Este es el primer nivel que vamos a ver(mencionados en la unidad 1). Con extencion se refiere a dos cosas del mundo de los objetos.

Pasos de un buen disenador

1. Reunir los requisitos del sistema
2. Crear esquema conceptual
3. Implementar la base de datos con algun DBMS(comercial o no)
4. Diseno fisico de la base de datos(indices)
5. Carga de datos y examinacion de funcionamiento

Sirve para modelar el mini-universo a representar, maneja tres nociones basicas:

- Conjunto de entidades
- Conjunto de relaciones
- Atributos

Entidades(entity type)

Una **entidad** es cualquier cosa del mundo real(concreta o obtracta) que posee existencia independiente. Debe poder saber si son la misma o no, entonces el cliente debe poder explicar que separa una entidad de otra(cuales atributos diferencian una entidad concreta de otra). Siempre hay que preguntar pues las reglas del negocio son arbitrarios y no se puede asumir nada.

Entonces, un **conjunto de entidades** es la coleccion de todas las entidadess que poseen los mismos atributos(no hablamos de sus valores).

```
Entidad -> Extension
```

```
Conjunto de entidades -> Esquema(intension)
```

Como minimo debe haber un atributo **identificatorio/clave/discriminatorio**. Si hay mas de uno, hay que marcarlos a los dos como claves.

Los que no son claves, se denominan **descriptores**.

Relaciones(Relationship type)

Una **relacion** es una asociacion entre entidades particulares. Mientras que un **conjunto de relaciones** es la coleccion de todas las relaciones del mismo tipo.

Formalmente, un conjunto de relaciones R es un subconjunto del producto cartesiano de conjuntos de entidades $E_1 \times E_2 \times \ldots \times E_n$ con $n \geq 2$

Caracteristicas

- **Grado o Aridad**: Cantidad de conjuntos de entidades participantes.
- **Rol**: Funcion que desempena una entidad en una relacion(generalmente no se rotula).

Restricciones

- **Recursividad**: Esto hace obligatorio rotular el rol. El mismo tipo de entidad participa **mas de una vez** en el conjunto de relaciones con **distintos roles**. Se consideran unarias, por ejemplo, es jefe.
- **Cardinalidad**: Numero de relaciones en que una entidad(concreta) puede participar, no confundir con aridad. Indica el tope superior, puede ser: 1:1(uno a uno), 1:N, N:1, M:N.

Participacion o existencia Especifica si la existencia de na entidad depende o no de su participacion con otra entidad en una relacion.

Un conjunto de entidades A participa **totalmente** en un cojnunto de relaciones R si todas sus entidades tienen que participar por lo menos de una relacion. Si no fuera asi, sera participacion **parcial**.

Atributos

Cada propiedad que describe una entidad se denomina **atributo**. Una entidad en perticular toma valores para cada una de sus atributos.

Cada atributo posee un rango de valores posible, se conoce como el dominio.

Los conjuntos de relaciones tambien pueden poseer atributos. Pero hay que tener cuidado, si el conjunto de relaciones es M:N, entonces, los atributos deben ser especificados como parte del conjunto de relaciones. Es un conjunto de relaciones con cardinalidad 1:N o 1:1 los atributos pueden migrarse hacia alguno de los conjuntos de entidades participantes.

> **Nota**: **No** van claves en la relacion.

Clasificacion

- Simple - Compuesto(sueldo es simple, direccion es compuesto pues es calle, numero, dpto).
- Univaluado(unico valor) - Multivaluado(por ejemplo, un auto, tiene mas de un color al mismo tiempo).
- Almacenado - Derivado(antiguedad puede derivarse de fecha_ingreso)

Claves Una entidad es **fuerte** si arma su clave a partir de sus atributos. Una entidad es **debil** si es parte del conjunto de entidades que tiene su clave formada por atributos propios mas atributos de otros conjuntos. Parte la tiene una entidad con la que se relacionea.

Si es debil, **automaticamente** tiene participacion total.

Modelo ER extendido agrega jerarquias

El modelo basico puede extenderse permitiendo representar mejor el mini-universo en cuestion. La extension permite expresar que un conjunto de entidades es subclase de otro co njunto de estidades debido a que posee mayor especializacion. Por ejemplo, *empleado* es superclase de los conjuntos *efectivo*, *contratado*, *free_lacne*.

Importante, cuando se verifica especializacion encontramos que el conjunto de entidades subclase hereda todos los atributos y participa de todas las relaciones de la superclase(yo que es un caso especial).

Tipos de jerarquias

Generalizacion Si se tiene un conjunto de entidades y se lo divide en subclases disjuntas.

El diagrama que representa esto se lo hace representando a cada una de las subclases y uniendolas a la superclases por una doble flecha, via un hexagono que posea el atributo que produjo la separacion.

Es-un Si sólo se especifica especialización, pero falla que al unir todas las subclases se obtenga la superclase, o se tiene que las subclases no son necesariamente disjuntas. El diagrama que representa esto es similar al anterior pero sin el hexágono del medio.

Semantica de relaciones

Redundantes o derivadas Se dice que una relación es redundante o derivada si al eliminarse no se pierde información. La información puede seguir obteniéndose.

A simple vista, podría pensarse que si un EER que se representa por medio de un grafo posee un ciclo hay relaciones redundantes. Sin embargo, que exista un grafo con ciclo no significa que haya necesariamente relaciones redundantes.

Las relaciones redundantes DEBEN eliminarse.

Binarias o ternarias Puede siempre reemplazarse una relación ternaria por varias binarias y mantener la semántica? No siempre. El reemplazo puede ocasionar pérdida de información.

> **Nota**: Ver representacion grafica de Teorey en apuntes de campus.

Unidad 3: Modelo de representacion/implementacion: Modelo relacional

Veremos el siguiente nivel de abstraccion, el de **implementacion**(logico). En la unidad anterior vimos el conceptual y el fisico lo vamos a ver muy por arriba.

Un poco de historia

El modelo(relacional) fue creado por Codd, investigador de IBM, en 1970. Ya existian otros modelos como las jerarquicas, pero se manejaba por medio de punteros y se navegaba de puntero a puntero buscando informacion, almacenada en disco. Tambien estaba el modelo de red, tipo grafo, pero similar.

Estos modelos anteriores se usaban mucho debido a su rapidez, lo cual era sumamente importante debido a la capacidad de las computadoras del momento. A pesar de que funcionaban, Codd proyecto para adelante

y se dio cuenta que a largo place iba a haber problemas frente a cambios en la estructura. De manera que, antes de la distribucion masiva de computadoras, penso en un modelo que se prepare para eso(estaba en IBM, conocia la tendencia).

El modelo se basaba en conjuntos(sustentado por teoria matematica), pero no era tan rapido como con punteros el uso de conjuntos, se dejo de lado la idea. Mas adelante, Codd propone el modelado grafico(entidad-relacion) y eso atrae atencion porque la representacion grafica permitio que sea mas facil de entender(antes era muy matematica). Poco despues surge la primer base de datos relacional, rapidamente reemplazando al jerarquico y al de red, siendo usado hasta dia de hoy por la mayoria de las empresas.

Modelo relacional

Representa todo el mini-universo por medio de relaciones en el sentido matematico(tuplas), no tiene tantos elementos como el entidad-relacion(entidad debil, fuerte, etc.).

Atributo

Tiene un **Dominio**. Indica valores atomicos posibles que puede tomar cierto atributo(String, integer, char). Pero se agrega un valor especial llamado `NULL`, **three value logic**. Significa que(alguno de estos): No aplica; Es aplicable, pero desconocido; Ni siquiera se sabe si es aplicable.

Ejemplo: Para representar una casa, puede ser lote, piso, numero, etc. Entonces, se coloca `NULL`, representando tres cosas al mismo tiempo.

Esquema

$$R(A_1.A_2, ..., A_n)$$

Esta dado por su nombre R y la lista de atributos A_1, ..., A_n donde cada A_i es el nombre del rol que juego algun dominio en dicho esquema. Varios atributos pueden tener igual dominio, pero cada uno tiene un rol diferente(nombre).

El esquema cambia con menos frecuencia que los datos, podria no cambiar nunca.

> **Nota**: En en el libro esta muy bien explicado

Relacion

Una relacion r es un subconjunto del producto cartesiano de los N dominios que definen R.

Dominios $D_1, D_2, ..., D_n \Rightarrow r \subseteq D_1 \times D_2 \times ... \times D_n$

Nota: Es imposible que una relacion tenga tuplas repetidas.

Cual es la maxima cantidad de elementos que puede tener una relacion(teniendo en cuenta que no se repiten)? Seria el producto cardinal de cada dominio, pero puede tener restricciones que lo hagan menor. Si no hay restricciones:

$$|D_1| \cdot |D_2| \cdot ... \cdot |D_n|$$

Normalmente nunca llegan al tope.

Por otro lado, las relaciones pueden verse como fotos instantaneas del mini-universo. Tiene dos partes:

- **Intension**: Encabezado con nombre de cada atributo interviniente.
- **Extension**: Cada fila o tupla muestra una coleccion de valores relacionados.

Relacional vs Entidad/Relacion

- El modelo entidad/relacion tiene conjunto de entidades, mientras que el relacional tiene un esquema de relaciones.
- El modelo entidad/relacion tiene conjunto de relaciones, mientras que el relacional tiene un esquema de relaciones.
- El modelo entidad/relacion tiene entidades, mientras que el relacional tiene un conjunto de tuplas en la tabla.
- El modelo entidad/relacion tiene relaciones, mientras que el relacional tiene un conjunto de tuplas en la tabla.
- El modelo entidad/relacion tiene una entidad, mientras que el relacional tiene una tupla de una tabla.
- El modelo entidad/relacion tiene una relacion, mientras que el relacional tiene una tupla de una tabla.

  Nota: La relacion en la relacional esta de forma implicita.

Notacion para el modelo relacional

Esto solo se usa para definiciones y demostraciones.

- **Esquemas de relacion**: R, S, T.
- **Atributos de un esquema de relacion**: A, B, C, A1, A2, ..., An.
- **Instancias de los esquemas**: r, s.
- **Valores de los atributos**: a, b, c, a1, a2, ..., an.
- **Filas o tuplas**: t, u, v, w(o con subindices).
- **Conjunto de atributos**: X, Y, Z o ABC en vez de {A, B, C}.
- **Restriccion de una tupla a un conjunto de atributos**: t[X] (o cualquier letra permitida). X seria el atributo que me interesa.

Restricciones del modelo relacional

Todas las restricciones hay que representarlas lo **antes posible**, es decir, si se puede representarlas en E/R, sino mantenerlas en el relacional. Son una de las principales ventajas de este modelo.

Restricciones de domino

Todo atributo debe ser atomico

Restricciones de clave

No puede haber tuplas repetidas en una relacion(garantizado por teoria conjuntista). Pero puede haber **menos** atributos que garantices la unicidad de las tuplas. Definicion formal de clave:

> Existe un subconjunto de atributos X dal esquema de relacion R, tal que toda instancia r verifica que si t1 y t2 son tuplas de dicha instancia y t1 $\neq$ t2 entonces t1[X] $\neq$ t2[X]. X es una **superclave** en este caso.

Mas economicamente, conviene tener una **clave candidata**, es decir, es superclave y minima respecto de la propiedad de ser superclave(si se remueve alguno de sus atributos deja de serlo). Es decir, aquella que me permite discriminar las entidades, pero minimal. Hace que sea mas barato verificar que un elemento no este repetido para la base de datos.

Restriccion de integridad de entidad

Ningun atributo que forme la clave puede ser `NULL`. Pues `NULL` es ambiguo, entonces no se puede usar para discriminar. Todos los motores que implementan este modelo siguen esta restriccion.

Restriccion de integridad referencial

Dependencia de inclusion, se especifica entre dos esquema de relaciones y se usa para garantizar consistencia entre ambas: **Clave Foranea(Foreign key)**. Un conjunto de atributos X de un esquema de relación R es una clave foránea de R si satisface las siguientes dos condiciones:

- Los atributos de X tienen el mismo dominio que alguna clave Y(candidate) de algún otro esquema de relación S
- Se verifica que siendo r una relacion de R y s una relacion de S, entonces para toda tupla t de r existe una tupla u de s tal que t[X] = u[Y] o bien t[X] es `NULL`.

(completar)

> **Nota**: Definicion formal en apuntes.
>
> **Nota**: Las proximas tres las vamos a ver mas adelante, despues del primer parcial

Restriccion de dependencias funcionales

...

...

Lenguajes del DBMS

- DDL: De definicion de datos, debe perimitirme crear….

Operacion de insercion

Se usa para gregar una tupla a una relacion, se debe proveer los valores de los atributos. El usuario facilmente puede violar alguna restriccion(de dominio, clave, integridad, y integridad referencial).

Ejemplo: Podria violar una restriccion de clave si quiere insertar una clave que ya existe(repetida).

Ejemplo: Podria violar restriccion de integridad referencial si se inserta una tupla cuya clave foranea refiere a una tupla que no existe en la relacion referida(si se aceptan `NULL` no habria porblema).

Operacion de borrado

Se usa para eliminar tuplas(pueden ser varias) en una relacion. Se debe proveer solo valores de los atributos que permiten identificar. Solo podria dar problemas en restriccion de integridad referencial, pero no en restriccion de dominio, clave, o integridad de entidad.

Que deberia hacer el DML si el usuario viola las restricciones:

- `ON DELETE NO ACTION`: Rechaza la eliminacion.
- `ON DELETE SET`: Permite borrado, pero modifica las claves foraneas que referencias las tuplas que se quiere borrar. Ponerles `NULL`, un valor default o pedirle al usuario que lo indique. La que vamos a usar.
- `ON DELETE CASCADE`: Borrar en cascada(recursivamente) todas las tuplas de las distintas relaciones que violen esta restriccion.

Operacion de modificacion

Se usa para cambiar los valores de ciertas tuplas(pueden ser varias) en una relacion. Se debe proveer los valores de los atributos que permiten identificarlas y los valores que se quieren cambiar.

> **Nota**: Para que lo pensemos(esta en apuntes de campus).

Pasaje de entidad/relacion a relacional

Pasos para un buen disenador

1. Reunir requerimientos del sistema
2. Crear un esquema conceptual
3. Implementar la base con algun DBMS comercial
4. Hacer diseno fisico de la base de datos(indices)
5. Implementar, cargar los datos necesarios y examinar el funcionamiento del sistema

Mapeo de entidades fuertes

En el caso de tener compuesto(los multivaluados se dejan para mas adelante):

1. Ir directamente a lo que son las claves atomicas, los compuestos se descomponen y se toman los valores, ignorando la compuesta.
2. Si el compuesto era clave, entonces sus valores pasan a formar la clave.

Mapeo de entidades debiles

Se colocan los atributos de la entidad y luego los atributos clave de la entidad con la cual se relaciona. Luego, la clave sera la clave debil mas los atributos clave agregador. Si tiene clave foranea, esta sera las claves agregadas. La foranea no pueden ser `NULL` pues son parte de la clave a nivel relacional y sera `ON DELETE CASCADE`.

Mapeo de atributos multivaluados

En una tabla aparte se coloca el multivaliado y las claves de la entidad. Donde la clave de la tabla serian todos los atributos. Sera `NOT NULL` y `ON DELETE CASCADE`.

Mapeo de relaciones

Binaria M:N

1. Crear un esquema de relacion para representar la relacion.

2. Agregarle los atributos atomicos o sus partes atomicas de los compuestos de la misma.
3. Incluir claves foraneas para que referencien…

Es imposible de embeber en alguna de las dos participantes, si o si hay que construir una relacion aparte que consiste de las claves(seran claves) y un atributo extra que es el de la relacion(en caso de tener).

Debido a las claves foraneas(las claves de una entidad son una y las claves de la otra son otra), entonces ambas seran `NOT NULL` y `ON DELETE CASCADE`. Si alguna entidad tubiera participacion total, no se podra representar con `ON DELETE CASCADE`(foreign key). Se necesita programacion avanzada que veremos mas adelante para resolver este caso.

Podria con esta tecnica representarse relaciones 1:N o 1:1? Si, pero es mas practico embeberlo en las relaciones existentes, evitar agregar mas tablas cuando sea posible por motivos de eficiencia.

Binarias 1:N No muy frecuente de encontrar, casos:

- Ninguna de las dos tiene participacion total, entonces hay que pensar en donde embebo la relacion. No hay opcion, se debe embeber en la entidad que sea N. Agregando a la tabla la clave de la que sea 1, sin olvidar el atributo de la relacion. Las claves de 1 obviamente son claves foraneas, pero pueden ser `NULL` pues la cardinalidad de la relacion es una cota superior.
- Alguna tiene participacio total, arranca igual, pero si la entidad con N tiene relacion total, entonces la clave foranea debe ser `NOT NULL` y `ON DELETE CASCADE`. Si la entidad con 1, tiene perticipacion total, se requiere programacion avanzada.

Binaria 1:1 Si ninguna tiene participacion total, podria embeberlo de cualquiera de las dos, es indistinto(FK `NULL`, `ON DELETE SELECT`?). Si se da el caso de participacion total, entonces necesito que la que tiene participacion total se borra si la otra tambien si borra(no tiene participacion total). Por lo tanto, FK es `NOT NULL` y `ON DELETE CASCADE`, caso contrario, se requiere de programacion avanzada(limite inferior).

Si ambas entidades tienen participacion total, debo poder garantizar que se borre en cascada, entonces ambas claves son FK es `NOT NULL` y `ON DELETE CASCADE`.

Finalmente, depende del programador, si realmente estan tan acopladas, conviene unirlas en una unica tabla, pues si borro una tengo que borrar la otra. Las nuevas claves seran las claves de las entidades, pero puede ser una sola la clave, no hay motivo para extender la clave.

Ternaria M:N:P No puede embeberse en ningun lado, debe ir a una relacion aparte, las unicas que pueden embeberse son las unarias y las binarias(menos M:N). Si o si, la nueva relacion va a tener las claves de todas las entidades, asi se mapean todas, pero difieren en la clave.

Como clave tiene la conbinacion, deja de tarea las FK.

Ternaria 1:M:N **Razonamiento**: Cuando es ternaria, cuando yo completo dejo un lado de libertad(en este caso significa combinar de a dos). Veo que no tiene sentido que la combinacion de todas las claves no puede ser clave, pues una clave de entidad 1 como mucho aparece una vez en esa combinacion. Por lo tanto, se deja afuera la clave de la entidad con 1 y la clave termina siendo las claves restantes combinadas.

Ternaria M:1:1 Similarmente, las claves seran las de la entidad con M y las de una entidad con 1.

Ternaria 1:1:1 Ahora, la clave sera la combinacion de las claves de dos entidad con 1, se puede elegir(dejar un **grado de libertad**).

Mapeo de Jerarquia generalizacion

Ver ejemplos graficos en apuntes.

Unidad 4: Lenguajes de consulta: Algebra y calculo relacional

Hay dos tipos de lenguaje de consulta:

- **Procedurales**: Se deben especificar las operaciones
- **No procedurales**: Se describe la informacion buscada

La mayoria de los lenguajes comerciales ofrecen una mezcla de ambos. El lenguaje de algebra relacional fue propuesto por el mismo Codd. Sirve para extraer la info de la BD.

Algebra

Hay dos categorias de operador:

- **Unario**: De una relacion R obtengo una relacion S.
- **Binario**: De R y R' se obtiene una relacion S.

De manera que los operadores binarios permiten anidar operaciones, y esto es muy importante.

Las relaciones del modelo relacional son conjuntos, entonces los operadores seran conjuntistas. Verifican la **propiedad de clausura** debido a que el dominio y codominio son conjuntos, es decir, no hay elementos repetidos. No tiene sentido que exista un operador que borre elementos repetidos.

Los operadores se pueden clasificar como:

- Fundamentales, muy pocos.
- Derivados, a partir de varios fundamentales(no potencial el lenguaje) se obtiene uno derivado, simplificando las expresiones.

Fundamentales

- **Seleccion**: Seleccioa subconjunto de tuplas de una relacion. $\sigma_{\text{condicion}}$ (nombre de relacion)
- **Proyeccion**: Selecciona cirtas columnas de la relacion original, seran menos tuplas si no se usa una clave(repetidos). $\pi_{\text{lista de atributos}}$ (nombre de relacion)
- **Union**: Entre relaciones compatibles, nunca se obtendran menos elementos que la relacion con menos elementos, mismas propiedades que union matematica. $r \cup s$
- **Diferencia**: No es conmutativa. r - s.
- **Producto cartesiano**: Se obtiene nueva relacion con una tupla por cada combinacion de tuplas de las relaciones originales, siempre tendra |r| * |s| tuplas. Un problema es que haya atributos repetidos(nombre), habra operacion de renombramiento. $r \times s$

Vamos a considerar que las relaciones no contienen el valor `NULL`, salvo que se indique lo contrario. Debido a que no hay forma de determinar si dos `NULL` son iguales por definicion.

Nota: Para union y diferencia deben ser **compatibles**. **Nota**: Un truco muy utilizado es usar $>$ o $<$ en vez de != para evitar que se repita la informacion(pero en distinto orden) al hacer producto cartesiano.

Renombramiento Se denota con ρ y se utiliza de la siguiente manera(X es el nuevo nombre):

$$\rho_X(\mathrm{E})$$

Simplemente cambia nombre de relacion. Por otro lado, se agregan parentesis y se pone una lista de atributos, misma cantidad que habia en E(vieja relacion) para renombrarlo:

$$\rho_{(\mathrm{X})}(\mathrm{E})$$

Sin embargo, en ocaciones especiales, se puede evitar el renombramiento simplemente agregandole un prefijo a la segunda aparicion del atributo repetido(al primero no se cambia). De manera que se agrega el nombre de la relacion '.' y el atributo.

Derivados

Recordar que no agregan nada, si no existieran solo llevaria mas trabajo realizar las **queries**, pero no agregan funcionalidad.

- Asignacion
- Division
- Interseccion(Similar a union, pero es derivada pues se puede obtener de restas)
- Junta
- Cociente

  Nota: Lo evalua en todos los examenes, muy potente.

Asignacion Permite asignar una expresion del algebra relacional en una variable temporal, se usa una flecha para denotarla. Permite definir operaciones que no son solo de consulta(insercion, borrado y actualizacoin), pero en esta materia no los vamos a usar. Por un lado, sin asignacion(ejemplo anidado):

$\pi_{\text{nombre, nota}}$ ($\sigma_{\text{alumno.legajo=examen.legajo}}$ (alumno $\times$ examen))

En comparacion con utilizar asignacion:

auxi <- $\sigma_{\text{alumno.legajo=examen.legajo}}$ (alumno $\times$ examen)

$\pi_{\text{nombre, nota}}$ (auxi)

Junta

- **Theta join**: Selecciona de todas las tuplas de un producto cartesiano aquiellas tuplas que verifican cierta condicion. r $\theta_{\text{condicion de junta}}$ s
- **Equijoin**: Parecido al theta, pero donde solo se aplica implicitamente =. En vez de $\theta_{\text{alumno.legajo=examen.legajo}}$ se usa simplemente θ_{legajo}. Sin embargo, no se usa mucho pues el resultado no saca columnas.

- **Natural join**: Tal cual esta en SQL, muy util. Realiza un producto cartesionano de las relaciones, aplica la seleccion forzada la igualidad de todos los atributos que aparecen en ambos esquemas de relacion, y luego se aplica la proyeccion para eliminar los atributos duplicados. Tiene un simbolo que no encontre en latex, pero es importante conocer todos los simbolos para el examen. Pide que haya coincidencia en los valores del producto cartesiano, si no hay atributos en comun es equivalente al producto cartesiano. r |x| s.
- **Outer join**: Variacion de natural join, permite que las tuplas que no cumplan con la igualidad se conserven, hay dos tipos: left, right y total. Cuando no coinciden, los atributos sobrantes van con `NULL`.
  - **Left outer join**: Se conservan las tuplas de r sin correspondencia en s, simbolo: r x| s.
  - **Rigth outer join**: Se conservan las tuplas de r sin correspondencia en r, simbolo: r |x s.
  - **Total outer join**: Se conservan las tuplas de ambos lados. r x|x s.

  Nota: La asignacion esta en un limbo, depende del autor se considerara derivado o fundamental. Incluso se debate si es un operador.

Cociente Sirve para las consultas que incluyen la expresion "para todo". S(el divisor) debe estar incluido en R. Se define, una tupla t esta en r % s ⇔

1. t esta en $\pi_{\text{R-S}}$(r)
2. Para toda tupla t_s de s hay una tupla t_r de r que cumple:
   - t_r[s] = t_s[s]
   - t_r[r-s] = t

Este operador es un buen ejemplo de porque se considera procedural, hay que tener mucho cuidado.

Definicion: Relaciones compatibles

Dos relaciones r(A1, A2, ..., An) y a (B1, B2, ..., Bn) son compatibles si tienen el mismo grado N y si dom(Ai) = dom(Bi) para todo i entre 1 y N.

> **Nota**: Para nosotros no tienen que tener igual dominio, sino que deben ser compatibles. Por ejemplo, `int` con `long`.

Calculo relacional

El lenguaje de consulta puro y no procedural esta basado en el calculo de predicados de primer orden de la logica matematica.

Hay dos tipos de calculo relacional:

- Calculo relacional de tuplas.
- Calculo relacional de dominios.

Calculo relacional de tuplas

Las queries tienen la forma:

{ T | formula(T)}

T representa las tuplas y la formula es la que deben verificar las tuplas de respuesta.

Ejemplo Final: Se quiere obtener los legajos de los alumnos que intentaron rendir todas las materias (aunque no las hubieran aprobado)

examen2			
legajo	NroActa	nota	nombreMateria
750	13.33	9	'Pgm I'
400	11.03	3	'BD'
400	11.04	7	'Pgm I'
400	11.08	8	'BD'
750	11.03	8	'BD'
2000	11.03	2	'BD'

materia
nombreMateria
'Pgm I'
'BD'

Rta: $\pi_{legajo,\ materia}$ (examen2)

$\pi_{legajo,\ materia}$ (examen2)	
legajo	nombreMateria
750	'Pgm I'
400	'BD'
400	'Pgm I'
750	'BD'
2000	'BD'

Figure 1: Ejemplo de uso de cociente

Atomos Hay dos formatos de atomos:

- r (V), variable(tupla) V pertenece a relacion r
- V[i] op U[j], comparar algun atributo de las variables.
- V[i] op c

Las variables que figuran en las formulas pueden ser libres(glbales) o ligadoas(locales). Las ligadas son las que estan dentro del alcance de un cuantificador existencial o universal(existe o para todo).

> Las unicas variables libres que pueden aparecer en una expresion del calculo relacional de tuplas son aquellas ligadas con un existe o un para todo.

Formulas Pueden ser:

- Atomo
- f1 $\vee$ f2
- f1 $\wedge$ f2
- $\neg$fi
- ($\exists$ U) (f_u)
- ($\forall$ U) (f_u)

Tipos

- **Seguras**: Devolver al usuario una consulta en un intervalo finito de tiempo y que este bien definida.
- **No segura**: Devuelve una relacion, pero esta es infinito / no esta bien definida. Entonces, no es interesante(como una especie de loop infinito).

  Nota: Prestar atencion a esto, las no seguras aparecen generalmente con negacion y cuantificador universal.

El **dominio activo** de una BS son los valores que estan presentes dentro de las relaciones, entonces formalmente una formula segura no puede formular consultas que extiendan el dominio activo de la BD.

Hay que usar leyes de logica para transformar una formula no segura a una segura, las que mas vamos a usar dos:

- ($\forall$ X) (p_X) se puede reemplazar por ($\neg\exists$ X) ($\neg p_X$)
- ($\exists$ X) (p_X) se puede reemplazar por ($\neg\forall$ X) ($\neg p_X$)

  Nota: Ver equivalencias entre algebra y el lenguaje en campus.

Es declarativo!

Calculo relacional de dominio

{ X1, X2, ..., Xn | formula(X1, X2, ..., Xn) }

Atomos

- r(X1, X2, ..., Xn)
- Xi op Xj

- Xi op c

Formulas

- Un atomo
- F1 ∨ F2
- F1 ∧ F2
- ¬F1
- (∃ U) (FU)
- (∀ U) (FU)

  Nota: Es valido poner (∃ X1, X2, ..., Xn) en vez de (∃ X1) (∃ X2) ... (∃ Xn)

Tipos Igual concepto que con las de tupla, solo cambia de tupla a dominio.

Unidad 5: Languaje de consulta SQL

Structures Query Language, creado por un gropu de IBM en 1974. Llamado originalmente SEQUEL, pero no habia convenciones ni reglas, esto se introdujo en SQL 1.

Sintacticamente se parece al calculo relacional, en particular al de tuplas.

SQL se ecnarga de todo destro de una DBMS, del diseno interno, vista y manipulacion. El concepto de portabilidad tambien aplica a bases de datos.

> **Nota**: Para el primer parcial va a entrar casi todo SQL 2.

Lenguaje de definicion de datos(DDL)

Involucra tipos, dominios y tablas.

Tipos

Hay que elegir el tipo de dato correcto para un atributo.

- Se restringen los valores.
- Se limitan las operaciones a realizar.
- Influye en el espacio que ocupara la tabla.

Esto garantiza la integridad semantica y permite representar las restricciones de dominio del modelo relacional.

SQL2 ofrece:

- `CHAR/CHARACTER(tam)`
- `VARCHAR/CHARVARYING/CHARACTERVARYING(max)` Estos pueden introducir **hops** dentro de la base, haciendo mas lenta la lectura(en la memoria no importa).
- `INTEGER/INT`
- `SMALLINT`
- `FLOAT`
- `REAL`

- `DOUBLE PRECISION`
- `NUMBER/DECIMAL/DEC`
- `DATE` yyyy-mm-dd
- `TIME` hh:mm:ss (lo menos compatible de SQL2)
- `TIMESTAMP` fecha y hora(como minimo 5 lugares para la reaccion de segundo
- `INTERVAL` permite sumar o restar algunos de los tipos anteriores

Dominios

Permite crear un dominio *a priori*, parecido al typedef de C.

```
CREATE DOMAIN nombre built_in [valor_default][restriccion]:
```

Sin embargo, no es muy compatible, asi que no lo vamos a usar.

Creacion de tablas persistentes

- Identificarla con nombre
- Indicar sus columnas
- Marcar sus claves
- Eventualmente, indicar claves foraneas y restricciones

SQL(a diferencia de un lenguaje verdaderamente relacional), permite que las tablas no tengan claves definidas, y por lo tanto permiten que las tuplas esten repetidas, en vez de sets tenemos BAGS.

Destruccion de tablas

```
DROP TABLE nombre_tabla [CASCADE|RESTRICT];
```

Cambio de esquema de tablas

- Agregar atributos
- Elimniar atributos
- Cambiar la definicion de un atributo
- Agregar restricciones
- Eliminar restricciones

Por ejemplo:

```
ALTER TABLE nombre tabla
    ALTER nombre_atributo DROP DEFAULT
```

Lenguaje de manipulacion de datos(DML)

Insercion de tuplas

Inserta una tupla por vez, mas tarde veremos insercion masiva.

```
INSERT INTO nombre_tabla [(lista_atributos)]
    values(lista_valores);
```

La lista se atributos solo se puede omitir si ingreso todos los atributos en la lista de valores.

Nota: Lo que esta entre corchetes es opcional.

Borrado de tuplas

Se eliminan tuplas ya ingresadas, la condicion puede ocasionar cierta accion por parte del DBMS (borrado en cascada).

```
DELETE FROM nombre_tabla [WHERE condicion];
```

No se puede usar `telefono = null`, sino se usa `is null`.

Actualizado de tuplas

Modifica tuplas ya existentes.

Lenguaje de manipulacion de datos (DML)

Usado para recuperar tuplas. Recordad que SQL no respeta todas las restricciones del modelo relacional.

- Elementos repetidos
- Tablas sin clave primaria

En general SQL no elimina repeticiones al buscar una tupla.

- No pierde tiempo.
- Resulta util cuando se usan funciones de agregacion.
- Es horrible cuando se ven reportes con filas repetidas.

SQL solo elimina repetidos cuando se usan operaciones conjuntistas:

- `UNION`
- `INTERSECT`
- `EXCEPT`

Explicitamente `ALL` mantiene duplicaciones, si se especifica.

Sintaxis:

```
SELECT [DISTINCT]
atrib1 [AS otroNombre1],
....
atribN [AS otroNombreN]

FROM lista_de_tablas
[WHERE condicion]
[GROUP BY lista_atributos_que_agrupan]
[HAVING condicion_para el grupo]
[ORDER BY lista_atributos]
```

Ambiguedades y soluciones

- Usar en el `FROM` mas de una tabla con el mismo nombre de atributo(usar tabla.atribRepetido).
- Usar en el `FROM` la misma tabla mas de una vez(usar alias).

Expresiones en la clausura `FROM`

Producto cartesioano

```
r CROSS JOIN s
```

```
(r CROSS JOIN s) AS
    nueva_relacion(lista_atributos_renombrada)
```

```
r , s
```

Junta natural Las tuplas que no matchean no aparecen en el resultado.

```
r NATUYRAL JOIN s
```

```
(r NATURAL JOIN s) AS
    nueva_relacion(lista_atributos_renombrada)
```

Tetha Join Es la junta default del SQL.

```
r [INNER] JOIN s ON condicion
```

```
(r [INNER] JOIN s ON condicion) AS
    nueva_relacion(lista_atributos_renombrada)
```

Semijunta natural o outer join

```
r LEFT OUTER JOIN s ON condicion
```

```
(...) AS ...
```

Tambien esta `RIGHT` y `FULL`.

Consultas anidadas

Pueden participar del `FROM`, `WHERE` y `HAVING`.

Expresiones en la clausura `WHERE`

Expresiones que se evaluen con `TRUE` o `FALSE`.

Operadores logicos

- `AND`
- `NOT`
- `OR`

Operadores relacionales

- $<$
- $<=$

- >
- >=
- <> (diferente)

Operador `LIKE` para strings Con comodin % matchea con cualquier substring, con _ matchea con cualquier caracter.

Operadores `IN`, `=ANY`, `=SOME` `IN` determina que un elemento esta dentro de una lista.

PostgreSQL no permite `=ANY` y `=SOME` con listas fijas.

Operadores `EXISTS` y `NOT EXISTS` Los vistos en logica.

> **Nota**: Los query nunca van a ser no seguro, pues no existe la negacion. El prompt siempre es positivo.
>
> **Nota**: `SET datestyle to DMY;`

Operadores sumarizacion

- `COUNT`
- `SUM`
- `MAX`
- `MIN`
- `AVG`

Estas funciones son para `SELECT` y `HAVING`, se aplican a un conjunto de valores y devuelven un solo valor. Algunas aceptan un atributo en el argumento. `COUNT`, `SUM` y `AVG` pueden aceptar `DISTINCT` como atributo.

Presentan varias restricciones:

- Solo en `SELECT` o `HAVING`
- No pueden anidarse
- Si aparece una funcio de agregacion en esas clausulas, todo atributo que aparezca en ella debe o bien ser agregado o participar de `GROUP BY`.
- La clausula `HAVING` no puede usarse si no hay `GROUP BY`.

> **Nota**: Menos `COUNT`, todas las funciones de agregacio no se aplican sobre valores `NULL`. Si se aplican sobre un conjunto vacio devuelve una unica tupla con valor `NULL`.
>
> **Nota**: `COUNT(nombreColumna)` no se aplica sobre atributos `NULL`(se descartan antes). Si se aplica sobre una tupla vacia devuelve una unica tupla con valor 0. Si es `COUNT(\*)` no se descartan los valores `NULL`.

`GROUP BY` Permite agrupar en base a alguna de las clausulas, `SELECT nombre, MAX(precio)` es valido en este caso si tiene un `GROUP BY nombre`, devuelve los maximos de cada nombre, pues agrupo los precios por nombre.

Aunque dos valores `NULL` no se pueden comparar, en el `GROUP BY` todos los `NULL` van al mismo grupo. No es trivial esto.

HAVING Sirve para filtrar en los grupos, se expresa siempre en funciones de agregacion. Un ejemplo:

```
SELECT trabaja_en
FROM empleado
GROUP BY trabaja_en
HAVING COUNT(*) >= 3
```

> **Nota**: Hay que conocer todas las opciones de cociente, nos puede preguntar cualquiera.

Transformacion de expresiones

Metodos o tendencias escalares:

- `CASE`: Transforma un escalar en otro. Devuelve el resultado al primer valor que verifica la expresion.
- `COALENSCE`: Transforma un escalar en otro. Devuelve el primer resultado que se evalue como `<> NULL`.
- `CAST`: Transforma un tipo escalar en otro.

> **Nota**: Importante recordar el order de operaciones. En esta clase no acerte ninguna, quede mal.

Modificacion

Para las tablas persistentes:

- Con la tabla destino preexistentes:

```
INSERT [Tabl
SELECT ...
```

Las tuplas insertadas en la tabla no tienen relacion con las tuplas de la tabla original.

- Creacion de una tabla a partir de otra:

```
CREATE TABLE nombre_tabla
(LIKE vieja_tabla)
```

No incluye PK ni FK, NOT NULL y defaulr. Solo genera columnas.

- Creacion de una tabla a partir de una consulta

```
CREATE TABLE nombre_table
AS (SELECT ...)
```

Se generan menos restricciones que la anterior, solo nombres de columnas y tipos. Esto se debe a que se esta generando a partir de un `SELECT` el cual hace simplemente un **dump**.

Para tablas temporarias:

- **Alcance dentro de la consulta**: Select anidados
- **Alcance dentro de la sesion**: Catalogadas o no cat(No la vamos a usar nosotros)

Vistas

Las vistan proveen un mecanismo de seguridad y autorizacion. La vista, o tambien llamada tabla virtual, toma algo de una o varias tablas que referencia a tablas subyacentes. En la vista no se copia, solo se referencia, de manera que si algo cambia en la tabla original, esto se vera reflejado en la tabla virtual.

```
CREATE VIEW nombre_vista [nombre_columnas]
AS select_que_la_define
```

Para borrarla se una `DROP VIEW`. No se puede creat una tabla virtual a traves de `SELECT` que contenga una clausula `ORDER BY`. Eso es normalmente, pero PostgreSWL si lo permite, solo hay que saber que no en todos los motores de base de datos funciona.

Cosas que **NO** se pueden hacer(restricciones):

- Actualizar un atributo derivado
- Actualizar una vista que se deriva de nua junta
- ActualizaSr donde hay funciones de agregacion
- Etc.(Depende del DBMS).

  Nota: Depende de la ordanizacion que dejo hacer o no a los distintos perfiles de usuario.

Permisos

El **DBA** siempre va por encima de todo, maneja permisos(ademas del owner). En SQL 2 se agrega la posibilidad de crear un SCHEMA;

Para crear areas separadas dentro de la misma base de datos, por ejemplo, en la del ITBA cada alumno tendra su namespace para hacer la practica, sino nadie podria trabajar ya que no se permiten nombres repetidos en la tabla.

```
SQL 2 CREATE SCHEMA nombre_esquema     AUTORIZATION nombre_usuario;
```

Si estoy trabajando sobre mis tablas, generalmente no se le agrega el prefijo, pero los demas que quieran usar mis tablas tendran que precisar con el namespace.

Hay dos niveles de privilegios en todos los DBMS(Ver apunte).

Es el algebra relacional equivalente al calculo relacional seguro?

Si, Codd lo demostro. Mas aun, definio que un lengiaje es **relacionalmente completo** si puede realizar todas las operaciones que tiene el algebra.

Por otro lado, Chandra y Horel tiene una definicin mas estricta:

> Un lenguaje es completo si es capaz de expresar exactamente el conjunto de todas las consultas computables(Turing-computable).

El calculo relacional no es ralacionalmente completo. Aho y Ullman mostraron que hay cierto tipo de consultas que el algebra relacional no puede computar, por ejemplo: Calculo de la clausura transitiva de una relacion.

Clausura

Una relacion que no cumple con la transitividad puede **clausurarse** para que si sea transitiva. Para lograr que SQL este resuelto hay que agregar una sentencia: la clausula `WITH`.]

Clausula `WITH`

Sirve para generar tablas temporarias

```
WITH [RECURSIVE] nombreRelacion(listaArgumentos) AS (ConsultaGeneradora
    <SELECT que accede a nombreRelacion>;
```

Sirve para no anidar tanto en el `FROM`. En SQL 3, el algoritmo recurrente sera:

```
WITH RECURSIVE
SuperiorDe(X, Y) AS
    (SELECT p1, p2 FROM JefeDe)
    UNION
    (SELECT Sup1.X, Sup2.Y
    FROM SuperiorDe Sup1, SuperiorDe Sup2
    WHERE Sup1.Y = Sup2.X)

SELECT X, Y FROM Superior De;
```

De no poner `RECURSIVE` no podria usar la tabla superior de adentro. En principio, si se implementa eso se consigue el algoritmo. Pero hay un problema, SQL 3 admite solo recursion lineal. De manera que la version final sera:

```
WITH RECURSIVE
SuperiorDe(X, Y) AS
    (SELECT p1, p2 FROM JefeDe)
    UNION
    (SELECT SuperiorDe.X, JefeDe.p2
    FROM SuperiorDe, JefeDe
    WHERE SuperiorDe.Y = JefeDe.p1)

SELECT X, Y FROM Superior De;
```

Unidad 7: Teoria de normalizacion

Hay varios tipos de anomalias:

- Anomalias de insercion(por ejemplo, insertar un alumno que todavia no rindio nada, pero tener que poner NULL en materias rendidas)
- Anomalias de modificacion(**no decir** que no se puede hacer el update, sino que decir que es un tema de performance ya que se encuentra en varios bloques y entonces hay que levantar un bloque, bajarlo y buscar otro…)
- Anomalia de eliminacion

El enfoque sintetico en la teoria esta bien, pero en la practica la ideal es unar un enfoque analitico. Es muy naive pensar que las cosas no cambian, podrian aparecer cambios en los cambios y en las tablas. El enfoque analitico esta alerta a posibles modificaciones.

Definicion: Dependencia funcional

Sean X e Y dos conjuntos de atributos de un esquema de relacion R. Se dice que X determina funcionalmente a Y o que Y depende funcionalmente de X, lo cual se denota X → Y, si se cumple:

$$\forall t_1, t_1 \in r : (t_1[X] = t_2[X] \Rightarrow t_1[Y] = t_2[Y])$$

F es el conjunto de dependencias funcionales de R.

Las reglas puestas por el usuario no se cuestionan.

Redefinicion: Superclave

Sea un esquema de relacion R, $X \subseteq R$ es superclave para R si se cumple:

$$X \to R$$

Redefinicion: Clave candidata

Sea un esquema de relacion R, $X \subseteq R$ es clave para R si se cumple:

1. Es superclave:
$$X \to R$$
2. Condicion minimal
$$\nexists Y \subset X / Y \to R$$

   Si bien ANCI pide dependencias funcionales, nadie lo obedece. Posgres no lo llego a implementar, lastima que no lo hace para UNIQUE, solo para PK.

Dependencia funcional

Clausura de F

El usuario me da las dependencias funcionales canonicas, pero hay un conjunto que se puede inferir. Se denota con F^+:

$$F^+ = \alpha \to \beta / F| = \alpha \to \beta$$

Hay algoritmos que lo definen. Para ello se usan los aximoas de Armstrong y las reglas de inferencia.

Con los aximos se pueden derivar todas las dependencias funcionales y no se encuentran de mas.

Axiomas de Armstrong

1. Reflexividad
2. Aumentacion
3. Transitividad

Reglas de inferencia

1. Union
2. PseudoTransitividad
3. Descomposicion(contrapuesta de la union)

   Nota: Las demostraciones las tenemos que saber usar.

Clausura de un conjunto de atributos

Si X es un conjunto de atributos de R, la clausura de X, denotada con X^+, respecto del conjunto de dependencias F de dciho esquema, es el cojnuto de atributos determinador por X a traves de las dependencias de F^+:

$$X^+ = A/X \rightarrow A \in F^+$$

Si hay atributos que no estan a la derecha de nadie, entonces pertenecen a **todas** las claves.

Algoritmo de Tableaux

Se arma una tabla con los esquemas que quiero separar, habra variables distinguidas y no distinguidas. (Ver apuntes para mas detalles).

Un buen algoritmo cumple:

- Sin perdida de informacion
- Preserva dependencias funcionales

Definicion: Descomposicion sin perdida de dependencias

Dado un esquema R, se dice que no hay perdida de dependencias al descomponerlo en N esquemas Ri, si sus clausuras de dependencias funcionales son equivalentes:

$$F'^+ = F^+ \text{ donde } F'^+ = \cup_{i=1}^{N} \pi_{Ri}(F^+)$$

Algoritmo para calcular la proyeccion de F^+ sobre un subesquema de R(el algoritmo)

(Ver apuntes)

Tema clase 6/10

Repaso

Si se calculan mal las claves, va a salir mal todo lo demas. Hay que practicar un monton porque el calculo de las claves tiene que estar perfecto.

Mal diseno

Un mal diseno tiene:

- Anomalia de insercion
- Anomalia de modificacion
- Anomalia de borrado

Enfoque analitico

> El enfoque sintentico lo propuso Codd

No es topdown, parte de entidad-relacion, aunque esto solo no garantiza que este libre de anomalias. Pues depende mucho del creador del modelo o de cambios en el negocio(cambios de reglas de negocios).

> Nunca quedarse con un solo algoritmo, tanto para cuando nos lo pidan como para tener plan B.

Se puede usar Tableaux para 2, pero generalmente conviene la otra forma.

> En el algoritmo para verificar las dependencias funcionales, es precondicion dejar a la derecha solo un atributo.

Proceso de normalizacion

Hay varios niveles niveles de descomposicion en esquemas mas pequenios sin anomalias. Funcionan asi: un nivel incluye a todos los niveles anteriores, siendo el quinto el nivel mas alto. En sistemas transaccionales(constantes inserciones, mod, etc.), necesitamos un esquema que este como minima en la tercera forma normal.

Para sistemas de toma de decisiones(no lo vemos), nos basta con segunda forma normal. Para sistemas de objetos relacionales se puede admitir priemra forma. En esta materia menos de 3 no. COmo se puede ver, depende mucho del contexto.

Usualmente, para llegar a la forma que a mi me interesa hay que pasar por las anteriores, pero Leticia tiene una forma de pasar a tercera sin pasar necesariamente por las anteriores.

Basados en dependencias funcionales

Las primeras 3 fueron propuestas por Codd, las otras(de las funcionales) fueron propuestas por Codd y Boyce.

Primera forma normal Un esquema de relacion R se encuentra en primera forma normal si sus atributos son atomicos.

Segunda forma normal Es si no existe atributo no primo en R que sea parcialmente dependiente de alguna clave(ver definicon de primo, son los que forman parte de alguna clave).

Recubrimiento minimal de F Es un conjunto Fm de dependencias funcionales que cumple ciertas propiedades(ver apuntes), deben estar en orden o no va a funcionar.

Tercera forma normal Un esquema de relacion R se encuentra en tercera forma normal si se cumple que

$$\forall \alpha \to \beta\text{: } \alpha \text{ es superclave o } \beta \text{ es primo}$$

(Ver algoritmo de descomposicion a tercera en apuntes)

Forma normal de Boyce-Codd (Ver apuntes)

Se procesa visualmente con un arbol, se parece un poco a la segunda forma normal. Es mas puro que tercera en cuanto redundancia, pero tiene un problema. El problema no es la perdida de informacion, pero si podria ocurrir que haya perdida de dependencias (si ocurre eso, hay que volver para atras y quedarse en tercera).

Basados en dependencias multivaluadas

Una dependencia multivaluada es, teniendo dos conjuntos α y β, la dependencia $\alpha \to\to \beta$ es multivaluada si para todo par de tuplas de la relacioni r que coinciden en todos los valores de los atributos de α, al intercambiar los valores de los atributos de β se obtienen tuplas que tambien son de la relacion.

> **Nota**: El concepto de trivialidad no es lo mismo

Axiomas **No son los mismos** que con dependencias funcionales. Tambien hay reglas que se derivan de los axiomas.

Cuarta forma normmal (Ver apuntes)

Es similar a BCNF,

Errores en el DER Clase de errores q ue ocurre cuando el analista no entiende a fondo los requisitos o ocurrio un cambio de negocio.

Basados en Dependencias de junta

(Ver ejemplos en apuntes)

Quinta forma normal Toda dependencia multivaluada puede expresarse en junta.

Se puede usar tableau, mientras cambian las variables van produciento una valuacion $/ro$(T).

Dos tableau son equivalentes si tienen la misma valuacion. Existe chase(T) respecto de CR.

(Ver metodo y teoremas en apuntes)

Unidad 8: SQL avanzado

SQL es muy declarativo, se pueden hacer muchas cosas con una sola sentencia, pero hay muchas cosas que no tiene(iteradores, variables). Es preferible usar SQL, pero cuando hay algo que no se pueda hacer conviene usar otro lenguaje.

Cuando se mezclan muchos lenguajes es muy dificil realizar migraciones:

- Dificil de migrar

- Los programadores usan sus lenguajes predilectos

Entonces, el ANSI agrego sentencias procedurales para que usen dentro del DBMS lo que deseaban: "PSM". Ocurrio que cuando ANSI especifico los estandares, muchos DBMS no lo obedecieron pues ya tenian su version hecha(ANSI tarda mucho en sacar estandares). Por lo que mucho de lo que pide ANSI esta implementado por muy pocos DBMS.

> Esto suele pasar con ANSI, nunca propone antes, sino que se une a las ideas que ya existen.

Persistent Stored Module(PSM)

Codigo compilado y almacenado en la base de datos que podra ser ejecutado si se tiene permiso de ejecucion. Tiene ventajas edicionales:

- Permite factorizar codigo
- Permite reusar codigo
- Se almacena en la base de datos(evita trafico entre cliente y servidor ya que no esta del lado del cliente).

El ANSI diferencia entre **funcion** y **procedimiento**. Los procedimientos difieren de functions pues no pueden invocarse desde SELECT ni FROM ni WHERE pues acepta cambiar sus parametros, es decir, sus parametros pueden entrar vacios y volver con otro valor(IN/OUT/INOUT).

PostgreSQL

No respeto nada de ANSI, pues se adelanto bastante y cuando vino ANSI ya estaba hecho todo y los programadores ya se habian adaptado.

Solo maneja `FUNCTION`, si retorna void podria considerarse como procedimiento. Los parametros pueden ser IN/OUT/INOUT, pero solo se pasan por valor, nunca vuelve un parametro modificado. Entonces, los INOUT o OUT se manejan devolviendo un parametro nuevo en la salida.

- **Un solo `OUT` o `INOUT`**: Genera un `RETURNS` con eso, en este caso no tenemos que poner un `RETURNS` explicito.
- **Mas de uno**: Genera un `RETURNS RECORD`(tipo compuesto, como struct de C)
- **Si no hay**: Obligatoriamente tengo que declarar un `RETURNS` de lo que quiero devolver. Por ahi habria que decolver un `VOID` explicitamente

Permite "no cablear y parametrizar:

- Variables
- Parametros formales

Muy util para debuggear son los **bloques anonimos**(especifico de PstgreSQL y de Oracle). Entonces, se puede utilizar para probar PSMs con los `raise notice...`, un analogo a `printf`.

Como se manejan los errores

Esto es tipico de una DB, no se puede comparar con C o Java. ANSI considera que hay dos tipos de errores:

- **Condicion**: Esperables o no graves
- **Excepcion**: Graves

Los errores se manipulan en una zona llamada SQLCA, ahi ocurre la comunicacion entre DBMS y aplicacion. Se usan las variables SQLCODE y SQLSTATE, su combinacion puede indicar distintos estados.

Porque los separa el ANSI?

Si se llega a producir una **condicion** en el codigo y no hay declarado ningun manejador, la base de datos entiende que no era nada grave y continua como si nada. Caso contrario, cede el control para que se maneje la excepcion.

Si se produce una **excepcion** y no hay manejador, termina el PSM y retorna al invocador(sigue en casada). Si no hay manejador, el motor no sigue de largo y termina **mal**.

Sin embargo, PostgreSQL no diferencia entre condiciones y excepciones, trata como excepciones. Si no hay manejador, el DBMS siempre va a terminal anormalmente(mal).

> Para aprovechar las excepciones ya definidas, hay que usar los numero que no estan en uso.

Algoritmo para manejar excepciones

Muy importante, no podemos meternos dentro de un motor de bases de datos, hay que conocerlo bien.

1. Deshace todas las sentencias SQL que hubiera realizado el(hace rollback).
2. Se fija si hay manejador de excepcion para ella en dicho bloque y procede con alguna de las siguientes 3 alternativas, segun el caso:
   - Si lo hay, entonces le transifere el control y la ejecucion sigue normal a partir de alli.
   - Si no lo hay y el bloque esta anidado en otro bloque, relanza la excepcion en el padre.
   - Si no lo hay y no esta anidado, relanza la excepcion y termina abnormalmente.

   Se aconseja no relanzar las excepciones hacia afuera, cada bloque debe ser responsable de intentar solucionar los errores que en el se produzcan.

Cursor

Una forma sintactica para referenciar una tupla por vez cuando se tiene una coleccion. Hay dos tipos:

- Cursor implicito
- Cursor explicito

Implicito

Cuando el `LENGTH(RESULTSET) <= 1`. Se usan en funciones de agregacion siempre que no haya `GROUP BY` y cuando se usa `WHERE` con `PRIMARY KEY`.

Si se intenta de usar con mas de una tupla, PostgrSQL devuelve la primera que aparece y no lanza error(ORACLE si).

Explicito

En cualquier otro caso, incluido el anterior. No hay manejador de exception NOT FOUND, sino que es una variable que hay que chequear como el errno de C. Es importante chequear la variable luego del `FETCH`.

Este tipo de cursor tambien puede modificar tuplas.

```
EXIT WHEN NOT FOUND
```

PostgreSQL tiene algunos atributos asociados a los cursores:

- `%FOUND`
- `%NOTFOUND`
- `%ROW_COUNT`

  Hay que tenes cuidado y no abusar de PSM, a veces se puede hacer de manera mas simple.

Codigo activo

Ahora exite la posibilidad de poner **codigo activo**. Antes el modelo relacion tenia varias restricciones:

- Todo atributo atomico
- Toda relacion debe tener clave
- …

Por otro lado, SQL2 tambien tenia restricciones.

Restricciones de dominio

Son tipo de atributo, valor default. Se puede agregar un `CHECK` dinamico o estatico.

Que pasaria si tengo `DEFAULT 100` y `CHECK(sueldo > 500)`?

Nada, solo falla en insercion o modificacion.

Para evitar errores PostgreSQL solo permite hacer `CHECK`s estaticos. ddfjlsdkfgjieei

Debe haber solo una `PRIMARY KEY` y el resto debe ser `UNIQUE`, aunque podrian ser todas `UNIQUE NOT NULL`. Por lo tanto, hay un motivo por el cual muchos programadores prefieren tener un `PRIMERY KEY`. Esto es porque es mas sencillo para usar como clave foranea, ya que nos evita escribir `ON DELETE...`.

Restricciones avanzadas

1. **Check de tabla**: Se chequea cierta condicion que no versa sobre un solo atributo.
2. **Assertion**: Garantiza consistencia en **todo momento** y la primer aparicion de codigo activo sin contar las claves foraneas. Son mas potentes que `FOREIGN KEY`.

   PostgreSQL no posee Assertions. Las reemplazan por Triggers.

Triggers

Mas complicado en sintaxis que los Assertion, pero es mas potente. Hay que pensarlo asi: si algo falla, se deshace todo lo asociado al trigger.

Un elemento activo esta del lado del motor, no tiene ida y vuelta. Permite que ciertas acciones sean ejecutadas automaticamente por el DBMS ante cierto evento(insert, delete, update, etc.).

Basicamente, un evento transfiere el control al codigo del trigger, luego hay un antes y un despues del evento en el cual se puede ejecutar el codigo de trigger en alguno de los dos. Tambien se podria chequear una

condicion opcional(se podria hacer un if) que acciona un codigo o no. Tambien se puede especificar como se va a ejecutar el codigo. Entonces, para un trigger se puede especificar:

- Timing
- Condicion
- `FOR EACH` o una sola vez(default).

  Ante un error se deshace toda la ejecucion del trigger, es lo mismo que con las acciones.

En la version de PostgreSQL se tomo la opcion `INSTEAD OF` para las vistas, se reemplaza la vista por lo que yo diga.

Tambien, a diferencia de ANSI, el codigo no va embebido, si o si hay que invocar a un **Trigger PSM**. No es un PSM cualquiera, tambien tiene que retornar un Trigger(depende de lo que retorne, como termina todo).

Trigger PSM

No tiene argumentos, tiene variables que le llegan por entorno:

- Old
- New
- TN_NARGS y TN_ARGV[]

Como debe retornar?

row-level Timing before o instead of:

- **Normal**: Devolver <> NULL.
- **Anormal**: Todo rollback, lanzar excepcion.
- **Normal, pero salteando operaciones**: Esta fea la forma, pero no esta mal. Querian que no falle como lo hace ANSI. Deja que el usuario haga lo que quiera, pero logeando la informacion. No va a ocurrir, pero al usuario no le salta la excepcion(si se quiere aumentar sueldo, por ejemplo).

Timing after:

- **Normal**: Devolver algo
- **Anormal**: Todo rollback, lanzar excepcion

Statement-level(no exite instead of)

- **Normal**: Devolver algo.
- **Anormal**: Todo rollback y lanza excepcion.

Restricciones adiciones

- Before o ofter solo puede ser usada con tablas.
- WHen no puede tener subqueries ni invocar PSM.
- No se puede incluir explicitamente commit(guardar la informacion) o rollback.
- No se deben lanzar triggers recursivos(cascading triggers).

Unidad 9: Programacion embebida - C

En este caso el lenguaje host no es el motor, en esta parte vamos a ver C con SQL embebido de manera estatic(podria ser de manera dinamica). En este caso no podemos usar GCC directamente, sino que vamos a usar un precompilador que es proveido por cada DBMS.

Los precompiladores agarran el codigo fuente y lo modifican ligeramente, luego se puede usar un compilador de C comun.

Host variable

Es una variable que usa el lenguaje C y tambien es usada por SQL, entonces para utilizarla en C se usa normalmente mientras que en SQL debera usarse como:

`:nombre`

Conexion con DB

Para ello hay que usar `CONNECT` y `DISCONNECT`.

Manejo de errores

Como puede C manejar los errores generados por SQL?

Forma implicita antigua

Usando el SQLCA permite comunicarse con la DB para saber si ocurrio un error. Permite leer `sqlcode` para determinar si la operacion fue exitosa.

Forma implicita moderna

Un alias a la forma anterior, solo hay que usar las variables `SQLCODE` y `SQLSTATE` en vez de un struct(pues el struct puede variar entre motores de bases de datos).

Forma whenever

Evita utilizar un `if` como en las opciones anteriores, se puede realizar el chequeo y accion **inline**.

> Es valido desde donde se lo setea hasta el final del lote fuente.

Basicamente, la declaracion se puede redefinir todas las veces que quiera, una declaracion es valida hasta que se declare la siguiente.

Variable indicator

Variable compartida entre el DBMS y la aplicacion cliente que se asocia a cada variable host que se sospeche pueda acarrear un problema.

Si esta en 0 entonces la variable host esta OK, si es -1 entonces hubo un NULL y no hay que usarla.

Cursor explicito

Como se podria hacer en programacion embebida en C? como `SELECT` no es embebible se debe utilizar `SELECT` con `INTO`.

Muy util usar:

```
EXEC SQL WHENEVER NOT FOUND DO BREAK;
```

Cosas a tener en cuenta

- Si hay que recuperar informacion ordenada pedirlo en el cursor: NUNCA ORDENAR EN EL CLIENTE.

Esto se debe a que el servidor tiene mucha capacidad de computo, no es el caso del cliente.

- Las variables compartidas(`host variables`) deben ser declaradas en funcion del tipo y size que tienen en la base de datos.

Podria ocurrir que el servidor este en 64 bits mientras que el cliente tenga 32 bits. No hay que asumir el size de los tipos pues sino ocurrira truncamiento.

> Todo motor de base de datos indica como cambia el size de los tipos dependiendo del entorno, hay que prestar atencion que variables y columnas se usaron.

Claves ficticias

Evitan usar los bytes que podria usar una clave foranea, es una clave que ocupa el size de un entero donde se trata de un id. Esto es especialmente util si la clave foranea es un arreglo de caracteres de 150 bytes.

Sin embargo, este tipo de desiciones no las realiza un programador, sino que se encarga el administrador de la base de datos pues se encarga de evaluar la informacion que se esta guardando.

Entonces, no es comun que un programador lo haga. Por otro lado, es un tradeoff, espacio v.s. performance.

Ahora, como definimos el identificador unico, como puede saber una tupla a que otra tupla se esta refiriendo. Obviamente no le vamos a pedir al usuario el numero. Una mala estrategia seria buscar el maximo id y agregarle 1, pues las bases de datos manejan concurrencia y habria condiciones de carrera.

Buena tecnica

Utilizar secuencia o valor autonumerico ofrecidos por los DBMS(todos los motores de base de datos ofrecen esto). PostgreSQL tiene todas las opciones, vamos a ver solo una: `SEQUENCE`.

Esto tiene un problema, si se borra una tupla con un ID quedaran huecos.

Otra opcion es usar `SERIAL`.

Todas estas opcionas garantizan un valor automatico, no repetido.

Pero, como se refencia desde otra tabla? La estrategia corresponde a buscar el numero con un `SELECT`.

> Es muy importante conocer las tecnicas: `INSERT RETURNING INTO`(evita realizar un `SELECT`).

Dos tablas se autoreferencian

Para la insercion hay que indicar que no **valide** la restriccion hasta realizar **commit**, y por supuesto siempre insertar conjuntamente en ambas tablas.

> Creacion de indices(Arbol B) sin necesidad de unidad debe ser considerada con cuidado.

Transacciones y programacion embebida

Una transaccion es un conjunto de sentencias, en vez de ejecutarse sueltas se deben ejecutar como un todo. Un buen ejemplo es la transferencia bancaria, debe ocurrir todo: Actualizar saldo en cuenta A, actualizar saldo en cuenta B.

En dicho caso, se realiza atomicamente utilizando un `COMMIT` y de deshace con un `ROLLBACK`.

> Una vez indicado el commit o rollback no es posible arrepentirse ya que se da comienzo a otra nueva transaccion.

Tienen dos cosas asociadas:

1. Garantizar consistencia de informacion
2. Permitir manejar concurrencia

   Todo o nada

   Los DBMS permiten trabajar con transacciones serializables que se comportan como si fueran seriales.

El problema de hacerlo serial es que no es interactivo, hay que esperar a que todo termine antes de empezar con otra cosa. Lo que hay que hacer es ir cambiando el nivel de aislamiento de las transacciones para permitir intercalar las trasacciones.

Otro tipo de transacciones es la serializable, donde el motor analiza lo que piden y las intercala hasta cierto nivel.

SQL2 y SQL3 agregaron la posibilidad de marcar una transaccion como `READ ONLY` o `READ WRITE` (default) con `SET TRANSACTION`. Ademas, se puede especificar el nivel de insolacion:

- `READ UNCOMMITED`
- `READ COMMITED`
- `REPEATEABLE READ`
- `SERIALIZABLE`

Problemas que soluciona la aislacion

- **Dirty read**: T1 actualiza un valor, T2 lo lee y luego T1 hace rollback. Quedando dirty read al read de T2.
- **Nonrepeatable read**: T2 lee un valor de un atributo, T1 lo actualiza/borra con commit y T2 vuelve a leerlo, obteniendo un valor diferente.
- **Phantoms**: Similar al anterior, pero en vez de update es insert. T2 lee tuplas y T2 inserta una nueva tupla con commit.

Nunca ocurrira alguno de estos programas si se es estricto y la base de datos es serializable. Por eso es el default, pero si se necesita performance se podria relajar las restricciones.

Nunca comenzar una transaccion cuando se abre una ventana de dialogo. Realizar transacciones atomicas.

Unidad 9: Programacion embebida - Java

Microsoft invento ODBC, una de las buenas cosas que hizo. Hizo un driver para los motores de base de datos(DB2, Orgacle, MySQL, etc.), de manera que cada motor implementa su driver para ODBC. Entonces, se obtubo una manera estandar de conectarse dinamicamente a cada motor.

El problema que los clientes que estaban pagando licencias de los motores para usar sus funcionalidades avanzadas no les gusto nada. De manera que no duro mucho esta propuesta por mas buena que parecio al principio.

Mas adelante surgio JDBC, basado en Java sigue la misma idea de ODBC. Registracion es lo que se conoce como elegir el driver deseado del pool.

Formas de cargar drivers

Primera forma

Se importa la clase manualmente en el codigo(estatica).

El unico problema que tiene es que hay que saber el nombre de las clases que implementa el driver. Entonces, solo se podria usar en un contexto interno, no para un cliente.

Segunda forma

Importando un string con el nombre del driver dinamicamente.

Tercera forma

Pasando el driver por linea de comando.

Cuarta forma

Por variable de entorno, esta ultima solo durante el desarrollo.

A partir de Java 8 ya no esta la indireccion JDBC-ODBC.

Tipos de drivers

Hay cuatro tipos y cada uno tiene un tipo de indireccion diferente.

Oracle y DB2 soportan todos los tipos, PostgreSQL soporta solo drivers tipo 3 y 4.

Conseguir estancia

Hay dos maneras de obtener un handle/estancia. JDBC siempre devuelve un cursor, no hay cursores implicitos(es mi responsabilidad hacer `next()`).

La ventaja de `executeQuery` con `execute` es la cantidad de lineas de codigo.

No usar executeUpdate

CreateStatement

No se pueden parametrizar los comandos, ya debe estar completo el string a la hora de ejecutar. Es muy completo y potente.

Sin embargo, tiene una desventaja, al no aceptar parametrizacion puede haber problemas de seguridad. Debido a que el string se puede armar concatenando, realizar SQL injection sea muy sencillo.

PreparedStatement

Permite parametrizar, se especifica el parametro con un `?`, luego es responsabilidad mia pasar los argumentos. Se pasa una sola vez el statement por la red y luego solo es cuestion de ir parando argumentos.

> No olvidar cerrar conexiones.

Siempre es conveniente usar `preparedStatement` a menos que haya un tipo de dato que todavia no se conozca(haciendo imposible pasarlo como tal salvo que se pase a string).

Nuevas versiones de JDBC

Una de las principales causas fue la aparicion de SQL3. Algunos cambios:

- Manejo de serial
- Manejo de blobs(datos de gran size)
- Invocacion de PSM o funciones no escalares(`CallableStatement`, no usar prepared o create)
- Simplificacion conexion(habia muchas maneras de especificar la direccion)

Pool de conexiones

En vez de darle una conexion a cada usuario, usar un pool de conexiones. Se especifican la cantidad de conexiones que aceptan, entonces se evita que se rompa el sistema o se sobrecargue. Entonces, si el pool esta lleno el usuario debe esperar, en caso de que el pool este siempre lleno se tendra que comprar mas capacidad para el sistema.